Connect Four with Minimax & Alpha-Beta Pruning

Julia Reynolds & Akiva Lippert

November 2026

Implementation Overview

- **Game Design:** The game is designed with Javafx, and is build to be modular. There are options to play against another human, against two preset AIs, and against an external AI that has been wired through a harness. You're also able to pit the AIs against each other and watch the games play out, which is very satisfying. It's a classic Connect 4 game, but the game board size has been implemented via runtime parameters, allowing the player/operator to adjust that between games during the same play session.
- **Minimax Implementation:** The two preset AIs were coded by each of us so we could both practice the algorithm. As such, we ended up implementing the core of the algorithms similarly. The Minimax implementation is done as explained in class, with the agent calculating paths assuming every other node will be optimal for the opponent.
- **Alpha-Beta Pruning:** The agents both prune the same way, and they evaluate columns from center outward. This biases the pruning towards boards with good center columns, since almost every winning board will have at least one of the agent's pieces in the center.
- **Heuristic Evaluation:** This was done by both AIs as explained in class: by comparing all windows of four on a game board. However, this is where the differences between the agents show up. When compared against each other, Akiva's agent is more aggressive, and puts more value on its own good positions than the opponent's pieces that make a bad position. Julia's agent is more defensive, penalizing a bad position more than it rewards a good one. Both agents gain a +3 for a center column piece, but since Akiva's agent has a smaller range of values in its heuristic function, those center column places are proportionally more valuable, and therefore bias the agent to play even more towards the center column compared to Julia's agent.

Scalability

The game's GUI includes a section with all the game settings and configurations.

Configurable Settings:

- **Rows/Columns:** Click the arrows on the spinners to adjust next to the indicated row/column to adjust the board size.
- **Players:** Select an option in the dropdown associated with each disc color to select the type of player. If an agent is selected, the depth, d, can be configured by clicking the arrows on the associated spinner. The player that gets the first move can be altered by selecting the color from the designated dropdown.

All values are capped at 12. Akiva's laptop was starting to struggle with rows, columns, and both agents' depths all set to 12, so he hard-capped it at that in the code

Experimental Setup

- **Computer:**
  - **OS:** Windows
  - **CPU:** Intel i9, 2.9GHz, 6 cores

  - **RAM:** 32 GB
- **Board Sizes:** 6x7, 8x9
- **Depths:** 2, 4, 6
- **Starting player:** Red (Human)
- **Number of games:** 3 per arrangement
- **Timing Method:** Differnece in System.nanoTime() for agent turn / 1,000,000

Results

As seen in 1, both AI's consistently beat the human player. As the search depth increased, so did the average move time and the maximum move time. More than three games were played for some of the configurations, but only the most recent 3 games of each configuration of the 6x7 board were included in the table and analysis for consistency. As demonstrated in 1, the game also works with a larger size board. In this case, it was an 8x9 board.

Table 1: Connect Four Agent Results

Board size	Search depth d	AI Agent	Games played	AI wins	Human wins	Draws	Average AI move time (ms)	Maximum AI move time (ms)
6×7	2	Julia's Agent	3	3	0	0	0	4
		Akiva's Agent	3	3	0	0	0	0
6×7	4	Julia's Agent	3	3	0	0	0.63	4
		Akiva's Agent	3	3	0	0	0	2
6×7	6	Julia's Agent	3	3	0	0	15.43	46
		Akiva's Agent	3	3	0	0	12	34
8×9	4	Julia's Agent	2	2	0	0	4.86	16
		Akiva's Agent	2	2	0	0	5.15	19

Discussion

- **Can your AI beat a human player? At which tested depths did it win?**
  Akiva: I played against my agent and Julia's agent, and couldn't win when the depth was ≥ 2.
- **How long does the AI take to select a move?**
  Akiva: Julia has the numbers worked up above, but the move selection depends on the depth and the viable paths after pruning. When I ran some 12-depth games, I had a few response times over 6 seconds. What was interesting (and fun!) was watching the response times drop as the game went on due to the alpha-beta pruning cutting away more and more non-viable paths.
- **How does increasing d affect response time and observed playing strength?**
  Akiva: I alluded to it above, but deeper depths leads to longer initial response times, especially on my potato laptop (not used for testing this time, sorry). A depth of 12 leads to very long searched through the tree, but the agent can pick much stronger moves.
- **How does increasing the board size affect performance?**
  Julia: As the size of the board increases, the performance decreases due to longer search times. As seen in 1, the average time for the 6x7 board with a depth of 4 was a little over 4 milliseconds shorter than for the 8x9 board with the same depth configuration.
- **What limitations or possible improvements did you identify?**
  Julia: The board size is limited to $4 - 12$ rows and columns to ensure it fits on a standard laptop screen. Discs shrink on larger boards so the window fits on screen. High search depths on large boards drastically increases the search time for the agent. Since there is no timeout, there is a noticeable wait while the agent thinks in these cases.